Tarea 2.2: Variables y operadores — Respuestas

Ejercicio 1. Evaluación de cortocircuito y precedencia

```
console.log(0 || "Fallback");       // "Fallback" -> 0 es falsy, así que el OR devuelve el segundo op
console.log(0 ?? "Fallback");       // 0 -> 0 no es null ni undefined, así que ?? respeta el 0
console.log(5 + "5");               // "55" -> number + string provoca concatenación
let i = 2; console.log(i++ * ++i);  // 8 -> i++ usa el 2 actual (i pasa a 3), luego ++i lo deja en 4
```

Ejercicio 4. Análisis de precedencia y operadores unarios combinados

```
let x = 3;
let y = 4;
let resultado = x++ * ++y + (x & 1);
```

Paso a paso:

1. `x++` (post-incremento): se usa el valor actual de `x` (3) en la operación, y **después** `x` pasa a valer 4.
2. `++y` (pre-incremento): `y` pasa a valer 5 **antes** de usarse en la operación, así que se usa el 5.
3. Multiplicación: `3 * 5 = 15`.
4. `(x & 1)`: en este punto `x` ya vale 4. En binario, `4 = 0100` y `1 = 0001`. `0100 & 0001 = 0000 = 0`.
5. Suma final: `15 + 0 = 15`.

Resultado: `resultado = 15`, `x = 4`, `y = 5`.

Ejercicio 5. Comparativa de cortocircuitos con valores falsy críticos

```
function fijarDescuento(descuento) {
  let tasaFinalA = descuento || 15;
  let tasaFinalB = descuento ?? 15;
  console.log(`Tasa A: ${tasaFinalA} | Tasa B: ${tasaFinalB}`);
}
```

Con `fijarDescuento(0)` se imprime: `Tasa A: 15 | Tasa B: 0`.

- `tasaFinalA` usa `||`, y como `0` es un valor **falsy**, el operador ignora el 0% legítimo del cliente VIP y lo sustituye por el valor por defecto (15). Esto es un error para este caso de uso.
- `tasaFinalB` usa `??`, que solo sustituye el valor cuando es `null` o `undefined`. Como `0` no es ninguno de los dos, respeta el descuento real del 0%.

La asignación correcta para este sistema es `tasaFinalB` (con `??`), porque un descuento del 0% es un valor válido y no debe confundirse con "no se ha indicado descuento".

Ejercicio tipo test

1. ¿Qué valor almacena `var miCoche;` sin asignar? → **b) undefined**
2. ¿Qué resultado da `12 + "5"`? → **c) "125"**
3. ¿Qué operador exige coincidencia en valor y tipo? → **c) ===**
4. ¿Qué devuelve `typeof null`? → **c) "object"**
5. ¿Qué operador asigna solo si la variable es `null` o `undefined`? → **c) ??=**
6. Con `const notas = [5, 8];`, ¿qué instrucción es legal? → **b) notas.push(7);** (modifica el contenido del array, no reasigna la constante)